

datalab0 报告

姓名：余旻恩

学号：2025202970

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse	logicalShift	leftBitCount	float_i2f	floatScale2	float64_f2i	floatPower2
37/37	1	1	2	4	4	3	3	4	4	4	3	4

解题报告

亮点

1. **bitXor** —— 只用 `~` 和 `&` 实现异或，7 个操作符，正好打满 `Max ops` 上限，推导过程中没有可以再省的地方。
2. **byteSwap** —— 第一版把 `n<<3 / m<<3` 直接写在表达式里，因为重复出现 6 次被计成 20 个操作符而超限；提成变量后降到 16。
3. **float_i2f** —— 返回值写成 `((exp-1)<<23) + frac`，利用 `frac` 自带的前导 1 在加法中自动进位，省掉了单独的指数修正和舍入溢出分支。

位运算题

bitAnd

其实就是数学意义上的德摩根律的直接应用： $x \& y = \sim(\sim x \mid \sim y)$ ，4 个操作符（上限 7）。

bitXor

两种处理夹杂在一块：只有取反和用按位和那么先考虑异或的特点也就是 01 得 1 和 00/11 得 0 那么这里要把不同给展现出来的话就要考虑取反看出区别 11/00→01 是一定的但是 01 可能变为 00/11 所以干脆再取一次 按位或即可 所以还要在造出来一个或上题已经说过了

1. 德摩根律： $x \mid y = \sim(\sim x \& \sim y)$
2. 异或展开： $x \wedge y = (x \mid y) \& \sim(x \& y)$

代入得 $\sim(x \& y) \& \sim(\sim x \& \sim y)$ 。它用了 4 个 `~` 和 3 个 `&`，共 7 个操作符，正好等于上限。

samesign

三种情况：符号位不同返回 0；符号位相同但一个是 0 另一个不是，返回 0；其余返回 1。这天然就符合按位异或的结构首先把最高位取出来看按位异或得 1 直接返回 0 但是得到 0 需要分类讨论 因为得到 0 的话最高位可以是 00 或 11（这个需要讨论），另外还有“一个值为 0 另一个不是”这种零星情况要单独处理，所以不能只判断符号位是否相同。

- 、
- 符号位用 `(x>>31)&1` 取（算术右移后最低位就是符号）。

• “恰好一个是 0”用 `(!x) ^ (!y)` 判断。0 既不正也不负，所以 `samesign(0,0)=1`、`samesign(0,-1)=0`，这一步是必须的。

8 个操作符（上限 12）。

logtwo

用二分法确定最高位的位置，等价于求 `floor(log2 v)`：这个真是非常精妙对于寻找最高有效位的部分直接二分法查找即可 其中要实现二分查找就来看 `s=(v>0xFFFF)<<4,v>=s,以及ans=ans|s`

- 依次判断 `v > 0xFFFF`、`> 0xFF`、`> 0xF`、`> 0x3`、`> 0x1`。
- 每命中一次，就把对应的位宽（16/8/4/2/1）左移进一个临时量并累加进答案，同时把 `v` 右移同样的位数，让下一轮只考察剩下的高位。
- 比较结果本身就是 0/1，直接当位移量用，省掉了分支。

5 轮刚好覆盖 32 位。18 个操作符（上限 25）。

byteSwap

三步：取出两个字节 → 把原位置清零 → 交叉写回。

- 取字节：`(x >> (m<<3)) & 0xFF`
- 清零：`x & ~(0xFF << (m<<3)) & ~(0xFF << (n<<3))`
- 写回：把第 `m` 字节放到第 `n` 字节的位置，反之亦然

（另有一种更短的写法是把“两个字节不同的位在两个位置同时翻转”当作交换，用异或掩码可以压到 11 个操作符；这里保留了更好读的提变量版本。）

本函数 16 个操作符（上限 17）。

reverse

循环 32 次，每次读 v 的最低位 ($v \& 1$)，挤进 ans 的最低位，同时把 ans 左移一位——相当于每读一位就把它放到对称的位置上。这个题目也很牛逼每次将对应的位数取出来的操作是 $v \& 1$ 再将 $v >= 1$ 右移一位把这一位塞回来 ans 对应的位数最后再将其往左边移动

7 个操作符（上限 30）。

logicalShift

C 的 $>>$ 对负数是算术右移，高位会补 1，要得到逻辑右移必须把这些补进来的位清掉。

掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ 的构造：这个构造可以直接把长度和对应的位置都给处理好真的很漂亮

- $(1 \ll 31) \gg n$ ：算术右移后有高 $n+1$ 位是 1
- 再 $\ll 1$ ：高 n 位是 1
- 取 $\sim$ ：低 $32-n$ 位是 1，正是要保留的部分

$n = 0$ 时整个式子退化成全 1 的恒等掩码，所以不需要为 $n = 0$ 写特例。6 个操作符（上限 20）。

leftBitCount

同样是二分法，这个题目更加牛逼，数的是左侧连续 1 的个数：

- 先看高 16 位是否全 1： $!(\sim(x \gg 16))$ 。是则答案加 16，并把 x 左移 16 位，把已经确认的全 1 部分移出考察范围。
- 再依次判断 8 / 4 / 2 / 1 位。

细节：1 位那一步只能判断到第 31 位，所以最后又补了一次同样的判断——这是为了处理 32 位全 1 的情况 ($\text{leftBitCount}(-1) = 32$)。

33 个操作符（上限 50）。

浮点题

float_i2f

分四步：

1. $x == 0$ 直接返回 0（它的位模式是全 0）。
2. 记符号位；负数用 $\sim ux + 1$ 取绝对值。
3. 用 $\text{while}(\text{temp} \gg 1)$ 数出最高位的位置 p ，等价于 $\text{floor}(\log_2 |x|)$ 。
4. 把有效位归一到 24 位： $p < 24$ 时左移补齐；否则右移丢掉多余位，并用多余部分 extra 与半值 half 比较做就近取偶舍入（大于半值进位，正好等于半值时看尾数末位是否为 1 决定）。

最巧的一处是返回值 $((\text{exp}-1) \ll 23) + \text{frac}$ ：

- frac 里包含着隐含的前导 1（它正好落在第 23 位）。把它加到指数域上时会自然产生一次进位，所以指数域写 $\text{exp}-1$ 才等价于标准的 $p+127$ 。
- 舍入导致 frac 进位到第 24 位时，这个加法也会自动把指数 +1，不需要额外分支。

27 个操作符（上限 30）。

floatScale2

按指数域分四种情况：

1. 指数全 1 (NaN 或 Inf)：原样返回，符合“NaN 时返回参数”的要求。
2. 指数为 0 (0 或非规格化数)：把低 31 位整体左移一位。这个写法同时兼容了“尾数最高位溢出后自动升格为规格化数”，不需要单独判断。
3. 指数等于 254： $\times 2$ 会溢出成 Inf，直接返回 $\text{sign} \mid 0x7F800000$ 。
4. 其余：指数域 +1，即加 $0x00800000$ 。

11 个操作符（上限 30）。

float64_f2i

把 64 位拆成符号位、11 位指数、高位 20 位尾数，然后：

- 先处理三种特殊值：指数全 1 返回 $0x80000000$ ； $\text{exp} < 1023$ （即 $|v| < 1$ ）返回 0（题目要求向零取整）； $E \geq 31$ 溢出返回 $0x80000000$ 。
- 补上隐含的前导 1 ($\text{high} \mid 0x100000$)。 $E < 21$ 时整数部分完全落在这 21 位里，直接右移 $20-E$ 位即可；否则先左移 $E-20$ 位，再拼上 uf1 里的高位。
- 负数最后取反。

18 个操作符（上限 60）。

floatPower2

直接按 x 的区间映射到三种结果：

x 的范围	含义	返回
$x < -149$	比最小非规格化数还小	0

x 的范围	含义	返回
$-149 \leq x < -126$	非规格化区间	$1 \ll (x + 149)$
$-126 \leq x \leq 127$	规格化区间	$(x + 127) \ll 23$
$x > 127$	溢出	$0x7F800000$ (+Inf)

9 个操作符（上限 30）。这里比预想的多 2 个：负数字面量 -149 、 -126 里的一元 $-$ 也会被计入操作符。

反馈/收获/感悟/总结

对于一些限制下的操作有一些是非常巧妙优雅的前面两题还好比较简单就是德摩根顶多配上一个异或的思考部分 但是交换字节也比较好想，对于取逻辑右移比较巧妙的是 $(1 \ll 31) \gg n \ll 1$ 这样可以避免直接 $(1 \ll 32)$ 的问题 最牛的就是对于两个用到二分查找的思想的地方有个是 $(x > 0xFFFF) \ll 4$ 另一个其实很难想是！ $(\sim(x > 16))$ 这个其实阐释了对应 0/1 的问题既有逻辑处理也有按位的巧妙结合 浮点数题目都比较正常就是操作上要明确